

# Laboratorio: Testing con JUnit 5

Ingegneria del Software - Traccia Esercizi: Classe Calculator *Tempo stimato: 90 minuti*

---

## Obiettivi

Al termine di questo laboratorio sarete in grado di:

- Scrivere test unitari con JUnit 5 per una classe Java
  - Usare le principali annotazioni: `@Test`, `@BeforeEach`, `@Nested`, `@ParameterizedTest`, `@RepeatedTest`, `@Disabled`, `@Tag`, `@Timeout`
  - Scegliere il metodo di assert corretto per ogni situazione
  - Strutturare una test suite leggibile e ben organizzata
- 

## Setup del Progetto

### Struttura delle cartelle

```
src/  
  main/java/  
    Calculator.java  
  test/java/  
    CalculatorTest.java
```

### Dipendenza Maven (pom.xml)

```
<dependency>  
  <groupId>org.junit.jupiter</groupId>  
  <artifactId>junit-jupiter</artifactId>  
  <version>5.10.2</version>  
  <scope>test</scope>  
</dependency>  
<dependency>  
  <groupId>org.junit.jupiter</groupId>  
  <artifactId>junit-jupiter-params</artifactId>  
  <version>5.10.2</version>  
  <scope>test</scope>  
</dependency>
```

### Classe da testare - Calculator.java

Questa è la classe che dovete testare. **Non modificarla.**

```

public class Calculator {

    public double add(double a, double b) {
        return a + b;
    }

    public double subtract(double a, double b) {
        return a - b;
    }

    public double multiply(double a, double b) {
        return a * b;
    }

    public double divide(double a, double b) {
        if (b == 0) {
            throw new IllegalArgumentException("Divisione per zero non consentita");
        }
        return a / b;
    }
}

```

---

## Riferimento Rapido - Annotazioni JUnit 5

| Annotazione        | Scopo                            | Quando usarla                |
|--------------------|----------------------------------|------------------------------|
| @Test              | Marca un metodo come test        | Ogni metodo di test          |
| @BeforeEach        | Eseguito prima di ogni test      | setUp(), init()              |
| @ParameterizedTest | Test con dati multipli           | Con @CsvSource, @ValueSource |
| @RepeatedTest(n)   | Ripete il test n volte           | Test di stabilità            |
| @Nested            | Raggruppa test in classi interne | Per organizzare per metodo   |
| @Disabled          | Salta il test                    | Con messaggio motivazione    |
| @Tag               | Etichetta per eseguire subset    | Con mvn test -Dgroups=...    |
| @Timeout           | Fallisce se supera il tempo      | In millisecondi o secondi    |

---

## Esercizi

### Esercizio 1 - Setup e primi test [Facile]

**Obiettivo:** creare la struttura base della test suite.

1. Create la classe `CalculatorTest` nel package corretto.
2. Aggiungete il campo privato `calc` di tipo `Calculator`.
3. Implementate un metodo `setUp()` annotato con `@BeforeEach` che istanzia un nuovo `Calculator` prima di ogni test.
4. Scrivete un `@Test` per verificare che  $2 + 3 = 5$ . Usate `assertEquals` con delta `1e-9`.
5. Scrivete un `@Test` per verificare che  $10 - 4 = 6$ .
6. Eseguite la suite e verificate che tutti i test passino.

**Hint:** Il delta in `assertEquals(expected, actual, delta)` serve a gestire l'imprecisione aritmetica dei `double`. Usate sempre `1e-9`.

---

### Esercizio 2 - Test su valori limite [Facile]

**Obiettivo:** testare casi limite e valori negativi.

1. Scrivete un test che verifica la somma di due numeri negativi.
2. Scrivete un test che verifica che qualsiasi numero moltiplicato per 0 dia 0.
3. Scrivete un test che verifica che  $X - X = 0$  per  $X = 42$ .
4. Scrivete un test che verifica la divisione con risultato decimale (es:  $1 / 4 = 0.25$ ).
5. Scrivete un test che verifica che dividere 0 per qualsiasi numero  $\neq 0$  dia 0.

**Hint:** Pensate: cosa succederebbe se il metodo fosse implementato male? Il vostro test lo catturerebbe?

---

### Esercizio 3 - Gestione delle eccezioni [Facile]

**Obiettivo:** verificare che le eccezioni vengano lanciate correttamente.

1. Usate `assertThrows` per verificare che `divide(5, 0)` lanci una `IllegalArgumentException`.
2. Verificate anche il messaggio dell'eccezione: deve essere esattamente "Divisione per zero non consentita".
3. Verificate che `divide(0, 0)` lanci anch'essa l'eccezione (e non restituisca 0 o NaN).

**Attenzione:** Non usate `try-catch` per testare le eccezioni: JUnit ha `assertThrows` esattamente per questo.

---

#### Esercizio 4 - Test parametrizzati con `@ParameterizedTest` [Medio]

**Obiettivo:** evitare la ripetizione scrivendo un solo test per più dati.

1. Scrivete un test parametrizzato con `@CsvSource` che verifica `add()` su almeno 5 coppie di valori (inclusi negativi e decimali).
2. Scrivete un test parametrizzato con `@CsvSource` che verifica `multiply()` su almeno 4 casi.
3. Usate `@ValueSource(doubles = {...})` per verificare che `divide(100, x)` non lanci eccezioni per una lista di divisori validi (es: 1, 2, -5, 0.1, 1000).
4. Personalizzate il nome del test con `name = "{0} + {1} = {2}"` per rendere il report più leggibile.

**Hint:** Ogni riga di `@CsvSource` genera un test separato nel report.  
Più righe = più copertura senza più codice.

---

#### Esercizio 5 - Organizzazione con `@Nested` [Medio]

**Obiettivo:** strutturare la test suite raggruppando i test per metodo.

1. Riscrivete (o copiate e riorganizzate) la vostra `CalculatorTest` usando classi `@Nested`: una per `add()`, una per `subtract()`, una per `multiply()`, una per `divide()`.
  2. Aggiungete `@DisplayName` a ciascuna classe nested per descrivere il gruppo in italiano.
  3. Ogni classe nested può avere il proprio `@BeforeEach` se necessario (oppure eredita quello esterno).
  4. Verificate che il report di test mostri la gerarchia corretta.
- 

#### Esercizio 6 - `assertAll`, `@Disabled`, `@Tag`, `@Timeout` [Medio]

**Obiettivo:** padroneggiare le funzionalità avanzate di `assert` e organizzazione.

##### Parte A - `assertAll`

1. Scrivete un test `verificaTutteLeOperazioni()` che usi `assertAll` per controllare in un solo test che `add`, `subtract`, `multiply` e `divide` producano i risultati attesi per gli stessi input (es: `a=6`, `b=3`).
2. Fate fallire intenzionalmente uno degli `assert` (cambiate il valore atteso) e osservate che `assertAll` continua ad eseguire tutti gli `assert` mostrando **tutti** i fallimenti insieme.

##### Parte B - `@Disabled` e `@Tag`

3. Aggiungete un test stub per un metodo `sqrt()` (radice quadrata) non ancora implementato. Annotatelo con `@Disabled` e un messaggio esplicativo.
4. Etichettate con `@Tag("fast")` tutti i test che non richiedono divisioni o eccezioni.
5. Etichettate con `@Tag("edge-case")` i test che verificano valori limite e zero.

### Parte C - @Timeout

6. Aggiungete `@Timeout(value = 500, unit = TimeUnit.MILLISECONDS)` a un test qualsiasi e verificate che passi.
7. Per capire cosa succede in caso di timeout, create un test temporaneo con `Thread.sleep(1000)` e `@Timeout(value = 100, unit = TimeUnit.MILLISECONDS)`. Osservate il fallimento, poi rimuovete il test.

**Hint:** `assertAll` è utile durante il refactoring: vedere tutti i fallimenti insieme è più efficiente che fixarne uno per volta.

---

### Esercizio 7 - @RepeatedTest e Assumptions [Medio]

**Obiettivo:** usare `@RepeatedTest` per stabilità e `assumeTrue` per test condizionali.

1. Scrivete un test con `@RepeatedTest(10)` che verifica che `add(3, 7)` restituisca sempre 10. Usate `RepetitionInfo` per includere il numero di ripetizione nel messaggio di errore.
2. Scrivete un test con `@RepeatedTest(5)` che verifica la stabilità di `multiply(-2, -3)`.
3. Scrivete un test che usa `assumeTrue(System.getProperty("os.name").contains("Linux"))` per eseguirsi solo su Linux. Verificate che venga **saltato** (non fallito) sulla vostra macchina se non siete su Linux.
4. Scrivete un test che usa `assumeTrue(false, "Sempre saltato")` e osservate il comportamento nel report.

**Hint:** La differenza tra un test che **FALLISCE** e uno che viene **SALTATO** è fondamentale: `assumeTrue` non indica un bug, indica che il test non si applica all'ambiente corrente.

---

### Esercizio 8 - Estensione della Calculator [Difficile]

**Obiettivo:** applicare tutto ciò che avete imparato su una classe estesa.

Estendete la classe `Calculator` con i seguenti metodi e scrivete i test relativi:

- `power(double base, int exp)`: eleva `base` alla potenza `exp`. Deve lanciare `IllegalArgumentException` se `exp` è negativo.

- `sqrt(double n)`: radice quadrata. Deve lanciare `IllegalArgumentException` se `n < 0`.
- `remainder(double a, double b)`: resto della divisione (come l'operatore `%`). Deve lanciare `IllegalArgumentException` se `b == 0`.

Per ciascun metodo dovete scrivere:

1. Almeno 3 test normali con `@Test`
  2. Almeno 1 test parametrizzato con `@ParameterizedTest` e `@CsvSource`
  3. Il test per l'eccezione con `assertThrows`
  4. Organizzate tutto in classi `@Nested`
-